

WEEK 8

Aim:

To design and implement a complete Online Food Ordering System named FoodEase using HTML, CSS, JavaScript, Node.js, Express.js, and JSON files for user, food, and order data management.

Description:

This experiment demonstrates the development of a simple full-stack food ordering web application. The frontend provides registration, login, a food dashboard, search and category filtering, cart management, checkout, and order history. Node.js with Express.js provides the backend REST endpoints, while JSON files are used as a simple database for users, foods, and orders. LocalStorage is used to maintain the logged-in user session and shopping cart in the browser.

Program:

Landing Page – Provides the entry point to the FoodEase application.

Registration – Accepts user details and stores new users in users.json.

Login – Validates username/email and password and creates a browser session.

Dashboard – Loads food items from foods.json, supports search/category filtering, and adds items to the cart.

Cart & Checkout – Manages quantities, calculates subtotal/delivery fee/total, and collects delivery and payment details.

Orders – Saves placed orders in orders.json and displays the logged-in user's order history.

Program Code:

- 1. app.js – Express server, JSON file handling, registration, login, food menu, order creation, and order retrieval.

```
const express = require("express");
const fs = require("fs");
const path = require("path");

const app = express();
const PORT = 3000;

app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(__dirname));

function readJSON(fileName, defaultValue = []) {
  const filePath = path.join(__dirname, fileName);
  try {
    if (!fs.existsSync(filePath)) {
      fs.writeFileSync(filePath, JSON.stringify(defaultValue, null, 2));
      return defaultValue;
    }
    const data = fs.readFileSync(filePath, "utf8");
    return data ? JSON.parse(data) : defaultValue;
  } catch (error) {
    return defaultValue;
  }
}

function writeJSON(fileName, data) {
  fs.writeFileSync(
    path.join(__dirname, fileName),
    JSON.stringify(data, null, 2)
  );
}
```

WEEK 8 – User Registration and Login

Program Code:

- 2. register.html – User registration and POST request to the Express backend.

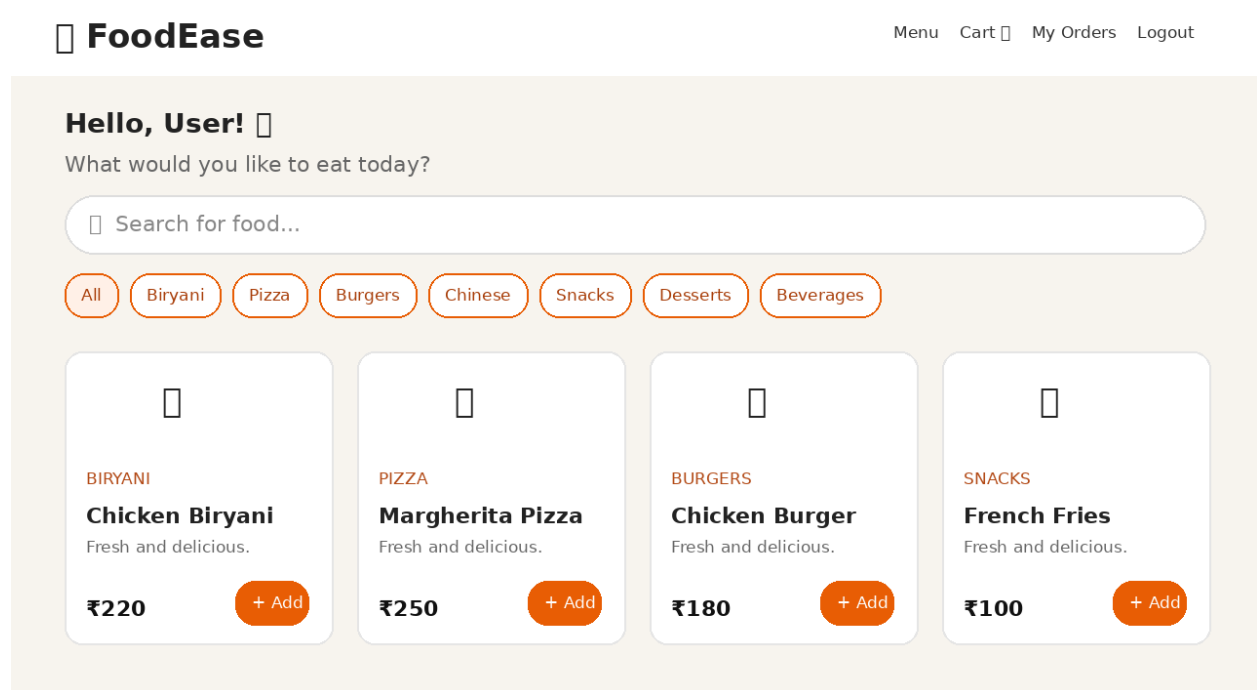
```
fetch("/register", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({
    name, email, mobile,
    username, password, address
  })
})
.then(response => response.json())
.then(result => {
  if (result.success) {
    alert("Registration successful!");
    window.location.href = "login.html";
  }
});
```

- 3. login.html – User authentication and session storage using LocalStorage.

```
fetch("/login", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({
    username: username,
    password: password
  })
})
.then(response => response.json())
.then(result => {
  if (result.success) {
    localStorage.setItem(
      "foodEaseUser",
      JSON.stringify(result.user)
    );
    window.location.href = "dashboard.html";
  }
});
```

Output:

The registration form accepts user details and stores them in users.json. After successful registration, the user is redirected to the login page. Valid login credentials open the FoodEase dashboard.



WEEK 8 – Food Dashboard

Program Code:

- 4. dashboard.html – Loads foods from the backend and provides search, category filtering, and Add to Cart functionality.

```
async function loadFoods() {
  const response = await fetch("/foods");
  foods = await response.json();
  displayFoods(foods);
}

function addToCart(foodId) {
  const food = foods.find(item => item.id === foodId);
  let cart =
    JSON.parse(localStorage.getItem("foodEaseCart")) || [];

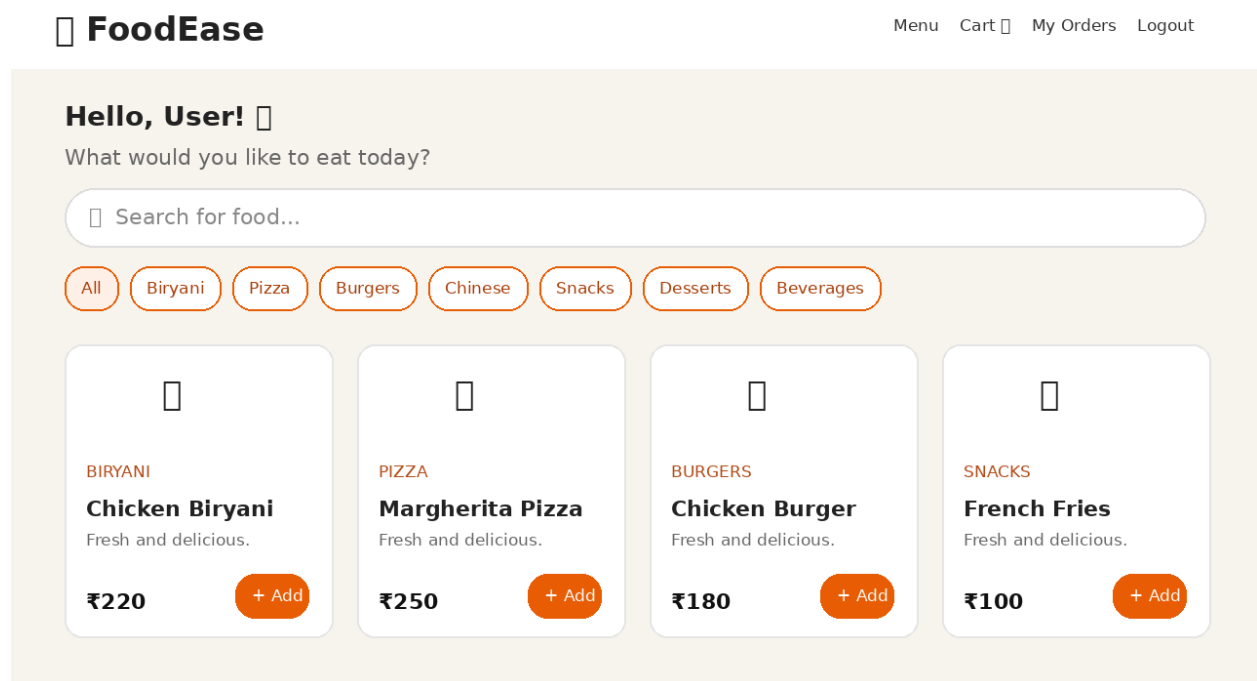
  const existingItem =
    cart.find(item => item.id === foodId);

  if (existingItem) {
    existingItem.quantity++;
  } else {
    cart.push({
      id: food.id,
      name: food.name,
      price: food.price,
      quantity: 1
    });
  }

  localStorage.setItem(
    "foodEaseCart",
    JSON.stringify(cart)
  );
}
```

Output:

The FoodEase dashboard displays food cards with category, description, price, and Add buttons. The search box filters food by name/category/description, while category buttons filter the menu.



WEEK 8 – Cart and Checkout

Program Code:

- 5. cart.html – Manages cart quantities, subtotal, delivery fee, total, delivery address, and payment method.

```
async function placeOrder() {
  const response = await fetch("/orders", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({
      username: user.username,
      items: cart,
      total: total,
      address: address,
      paymentMethod: payment
    })
  });

  const result = await response.json();

  if (result.success) {
    localStorage.removeItem("foodEaseCart");
    window.location.href = "orders.html";
  }
}
```

Output:



Menu Cart  My Orders Logout

Your Cart

Review your items before checkout

French Fries
₹100 each

- 2 +

₹200

Chicken Biryani
₹220 each

- 1 +

₹220

Order Summary

Items	3
Subtotal	₹420
Delivery Fee	₹40
Total	₹460

Proceed to Checkout

Checkout

Delivery Address

Visakhapatnam, Andhra Pradesh

Payment Method

☒ Cash on Delivery

☐ Online Payment (Demo)

Total Payable:

₹460

Confirm Order

The cart calculates the subtotal and adds a fixed delivery fee of ₹40. Checkout collects the delivery address and supports Cash on Delivery and Online Payment (Demo).

WEEK 8 – Order Management

Program Code:

- 6. orders.html – Requests the logged-in user's orders from the Express backend and displays order details.

```
const user =
  JSON.parse(localStorage.getItem("foodEaseUser"));

async function loadOrders() {
  const response =
    await fetch(`/orders/${user.username}`);

  const result = await response.json();

  if (result.success) {
    displayOrders(result.orders);
  }
}

loadOrders();
```

Backend order endpoints:

```
app.post("/orders", (req, res) => {
  // create and save a new order in orders.json
});

app.get("/orders/:username", (req, res) => {
  const orders = readJSON("orders.json", []);
  const userOrders = orders.filter(
    order => order.username.toLowerCase() ===
      req.params.username.toLowerCase()
  );
  res.json({ success: true, orders: userOrders });
});
```

Output:

My Orders

View your previous FoodEase orders

Order #1001

Confirmed

18/09/2026, 7:30 PM

French Fries	₹100 × 2	₹200
Chicken Biryani	₹220 × 1	₹220
Total Amount		₹460

After an order is confirmed, it is stored in orders.json and the user's My Orders page shows the order ID, date, items, total amount, and status.

WEEK 8 – Details About Program and Result

Details About Program:

Saved in: Week 8 folder of the repository.

Application Name: FoodEase – Online Food Ordering System.

Technology used: HTML, CSS, JavaScript, Node.js, Express.js, and JSON.

Frontend: HTML/CSS/JavaScript with Fetch API and LocalStorage.

Backend: Node.js with Express.js REST endpoints.

Data storage: users.json, foods.json, and orders.json.

Execution: The application is executed using the Node.js runtime through CMD/Terminal.

Server URL: <http://localhost:3000>

Main modules: Registration, Login, Food Dashboard, Search, Category Filter, Cart, Checkout, Order Placement, My Orders, and Logout.

Result:

The FoodEase Online Food Ordering System was successfully implemented using Node.js and Express.js with HTML, CSS, JavaScript, and JSON data files. The application supports user registration and login, food browsing with search and categories, cart management, checkout, order placement, and order history. The complete frontend-backend flow was tested through the Node.js server at localhost:3000.